

🌀 Kezdjük az alapokkal: **for** ciklus egy listán

Példa:

```
nevek = ['Anna', 'Béla', 'Csaba']  
for nev in nevek:  
    print(nev)
```

🔗 Ez azt jelenti: „minden elemre a listában, csináld ezt”.

🌀 **for** ciklus listák listáján

Példa:

```
vasarlasok = [['kenyér', 'tej'], ['alma', 'vaj'], ['tojás', 'kenyér', 'tea']]  
  
for vasarlas in vasarlasok:  
    print(vasarlas)
```

🔗 Kimenet:

```
['kenyér', 'tej']  
['alma', 'vaj']  
['tojás', 'kenyér', 'tea']
```

🌀 Bontsuk tovább: minden árucikk külön sorban

```
for vasarlas in vasarlasok:  
    for aru in vasarlas:  
        print(aru)
```

🔗 Kimenet:

```
kenyér  
tej  
alma  
vaj  
tojás  
kenyér  
tea
```

Magyarázat:

- A **külső for ciklus** végigmegy a vásárlásokon (azaz: az egyes listákon).
- A **belső for ciklus** végigmegy az adott vásárlás árucikkein.
- Így minden terméket külön-külön érünk el, akkor is, ha a szerkezet *listák listája*.

Mi az `enumerate()`?

Ez itt:

```
for i, nev in enumerate(nevek):  
    print(i, nev)
```

Kimenet:

```
0 Anna  
1 Béla  
2 Csaba
```

Tehát az `enumerate()` segít: **hányadik elemnél vagyunk + mi az értéke**.

Egy konkrét, hasznos feladat vele:

Feladat: Írd ki, melyik vásárlásban volt benne az adott termék!

```
vasarlasok = [['tej', 'kenyér'], ['alma', 'vaj'], ['tej', 'alma']]  
megadott_aru = 'alma'  
talalatok = []  
  
for i, vasarlas in enumerate(vasarlasok):  
    if megadott_aru in vasarlas:  
        talalatok.append(i)  
  
print(talalatok)
```

Kimenet:

```
[1, 2]
```

Ez azt jelenti: az „alma” benne volt az 1. és 2. vásárlásban.(ezek indexek, figyelni kell rá, ha a feladat sorszámot kér, akkor $i+1$!!)

Mikor használjunk szótárat?

Amikor **valamit össze akarunk számolni vagy csoportosítani szeretnénk**, hasonlóan SQL-ben a **GROUP BY-hoz**, akkor **a szótár a legjobb eszköz**.

 Tipikus feladatok, ahol szótárt érdemes használni:

1. Egy utcában hány ház van az egyes házzámokon?

```
hazszamok = [12, 14, 12, 14, 12, 16]
# → {12: 3, 14: 2, 16: 1}
```

```
statisztika = {}

for haz in hazszamok:
    if haz not in statisztika:
        statisztika[haz] = 1
    else:
        statisztika[haz] += 1

for haz, db in statisztika.items():
    print(f"{db} db ház a(z) {haz}-es számon")
```

2. Egy boltban egy terméket hányszor vettek meg?

```
termekek = ['tej', 'kenyér', 'tej', 'alma', 'tej']
# → {'tej': 3, 'kenyér': 1, 'alma': 1}
```

3. Egy feladatra hányan adtak választ?

```
valaszok = ['A', 'B', 'A', 'C', 'B', 'A']
# → {'A': 3, 'B': 2, 'C': 1}
```

4. Szavazásnál ki hány szavazatot kapott?

```
szavazatok = ['Juli', 'Peti', 'Juli', 'Juli', 'Peti']  
# → {'Juli': 3, 'Peti': 2}
```

5. Melyik tantárgyból hány pontot szerzett egy diák?

```
pontok = [('matek', 5), ('töri', 4), ('matek', 4)]  
# → {'matek': 9, 'töri': 4}
```

```
osszegzes = {}  
  
for tantargy, pont in pontok:  
    if tantargy not in osszegzes:  
        osszegzes[tantargy] = pont  
    else:  
        osszegzes[tantargy] += pont  
  
# Kiírás tantárgyanként  
for tantargy, osszpont in osszegzes.items():  
    print(f"{tantargy}: összesen {osszpont} pont")
```

👉 Ez már **összeadásra** is példa, nem csak darabszámolásra!

🌀 Összefoglalva :

Ha **valami többször is előfordul**, és szeretnéd **megszámolni**, hogy **hányszor**, akkor **szótárat** kell használnod.

A `items()` egy **szótár (dictionary)** metódus, amely lehetővé teszi, hogy a szótár kulcsait és értékeit **párokbán** érjük el.

🧠 Elméleti összefoglaló: `items()` metódus

- A `items()` metódus a **szótárban lévő összes kulcs-érték pár** listáját adja vissza.
- A kulcs-érték párokat **tupla** formájában kapjuk meg, tehát minden egyes elem egy **(kulcs, érték)** pár.

📄 Szintaxis:

```
dictionary.items()
```

- **Visszatérési érték:** egy **view object**, ami tartalmazza a kulcs-érték párokat.

Példa:

```
szotar = {'alma': 5, 'banán': 3, 'körte': 2}

# items() segítségével végigiterálhatunk a kulcs-érték párokon
for kulcs, ertekek in szotar.items():
    print(f"Kulcs: {kulcs}, Érték: {ertekek}")
```

Kimenet:

```
Kulcs: alma, Érték: 5
Kulcs: banán, Érték: 3
Kulcs: körte, Érték: 2
```

Fontosabb jellemzők:

1. **Kulcs-érték párok:** Az `items()` lehetővé teszi, hogy a szótár minden egyes kulcsa és az ahhoz tartozó értéke párban elérhető legyen.
2. **Iterálás egyszerűen:** Ha egy szótár kulcs-érték párain szeretnél végigiterálni, az `items()` az egyik legjobb módszer.
3. **Lehetőség további műveletekhez:** Az `items()` használatával például könnyedén rendezhetjük vagy szűrhetjük a szótár elemeit.

Példa: Rendezés kulcs szerint:

```
szotar = {'alma': 5, 'banán': 3, 'körte': 2}

# Rendezés kulcs szerint
for kulcs, ertekek in sorted(szotar.items()):
    print(f"{kulcs}: {ertekek}")
```

Kimenet:

```
alma: 5
banán: 3
körte: 2
```

Slicing – részlista kivágás

A **slicing** segítségével **részlistákat** tudunk készíteni egy listából. A **szintaxis**:

```
list[start:end:step]
```

- **start**: A kezdő index, ahonnan a listát kivágjuk (alapértelmezett: 0).
- **end**: Az utolsó index, ameddig a listát kivágjuk (alapértelmezett: a lista vége).
- **step**: Mekkora lépésekkel haladjunk (alapértelmezett: 1).

Fontos, hogy **az end index nem lesz benne** a kivágott listában, tehát a kivágás **a start index-től az end előtti index-ig tart**.

Példák:

1. Adott indexig szeretnénk megnézni az elemeket (pl. az első 3 elemet):

```
nevek = ['Anna', 'Béla', 'Csaba', 'Dávid', 'Eszter']

# Első 3 elem (index 0, 1, 2)
print(nevek[:3]) # ['Anna', 'Béla', 'Csaba']
```

- **Mit csinálunk itt?** Az első három elemet kérjük le. Az indexek **0, 1, 2** értékeit, de a 3-as index nem szerepel benne.

2. Egy adott indexig (de a legutolsót is beleértve):

```
# Első 4 elem (index 0, 1, 2, 3)
print(nevek[:4]) # ['Anna', 'Béla', 'Csaba', 'Dávid']
```

- **Mit csinálunk itt?** Az első 4 elemet kérjük le, tehát az indexek **0, 1, 2, 3** értékeit, és a 4-es index nem szerepel benne.

3. Slicing lépésközzel (minden második elem, pl. az első 6-ból):

```
nevek = ['Anna', 'Béla', 'Csaba', 'Dávid', 'Eszter', 'Fanni', 'Gábor']

# Minden második elem
print(nevek[::2]) # ['Anna', 'Csaba', 'Eszter', 'Gábor']
```

- **Mit csinálunk itt?** Az összes elem közül **minden második elemet** veszünk (indexek: 0, 2, 4, 6).

4. Részlista a lista végéből (pl. utolsó 2 elem):

```
print(nevek[-2:]) # ['Fanni', 'Gábor']
```

- **Mit csinálunk itt?** Az utolsó két elemet kérjük le. A negatív indexek a lista végétől számolnak hátrafelé, így -2 az utolsó előtti elem, -1 pedig az utolsó.

Feladatok — Slicing gyakorlására:

1. Feladat: Válaszd ki az első 5 számot a következő listából!

```
szamok = [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

- **Kimenet:** [1, 2, 3, 4, 5]

2. Feladat: Kérd le a lista utolsó 3 elemét!

```
nevek = ['Anna', 'Béla', 'Csaba', 'Dávid', 'Eszter']
```

- **Kimenet:** ['Csaba', 'Dávid', 'Eszter']

Fájlkezelés Pythonban: Megnyitás, Írás és Olvasás

A fájlba írás Pythonban egyszerűen elvégezhető az `open()` függvénnyel. Íme egy gyors összefoglaló a fájlírás módjáról:

Fájl megnyitása írásra:

A fájl megnyitásához az `open()` függvényt használjuk, amely két alapvető paramétert vár:

1. **Fájl neve:** A fájl elérési útja és neve (pl. `"output.txt"`).
2. **Mód:** Az írás módja. Ha írni szeretnénk, akkor `"w"` módot használunk, ami felülírja a fájlt, ha már létezik. Ha csak hozzá szeretnénk fűzni, akkor `"a"` módot használunk.

A `file` paraméter értékeként adhatod meg a megnyitott fájlt, így a print kimenete nem a konzolra, hanem a fájlba íródik.

Példa:

```
# Fájl megnyitása írásra (felülírja, ha létezik)
output_file = open("output.txt", "w", encoding="utf-8")

# Fájlba írás
print("Ez a sor a fájlba kerül.", file=output_file)
print("Ez is a fájlba kerül.", file=output_file)

# Fájl bezárása
file.close()
```

Rövid összefoglaló:

- Az `open()` függvény segítségével fájlt nyitsz meg, és az `encoding` paraméter segítségével megadhatod, hogy milyen karakterkódolással dolgozzon a program (pl. `utf-8`).
- A `file` paramétert a `print()` függvénybe átadva a szöveg nem a konzolra, hanem a fájlba íródik.
- A fájl írása után mindig **zárd be** a fájlt a `close()` metódussal--> (érettségin egyébként nem jár érte pontlevonás, de pl ha később szeretnél olvasni is az `output_file`-ből, akkor azt csak úgy tudod megtenni ha előtte bezártad a file-t!!)

random modul – Véletlenszerű választás Pythonban

A `random` modul segítségével egyszerűen generálhatunk véletlenszerű értékeket.

1. Véletlen egész szám – `randint()`

Ha **egy adott intervallumból szeretnénk véletlen számot kapni**, akkor a `randint()` függvényt használjuk:

```
random.randint(1, 10)
```

Ez **egy egész számot ad vissza** a megadott tartományból (beleértve a határokat is!!).

2. Véletlen elem kiválasztása listából – `choice()`

Ha **egy listából vagy szövegből** szeretnénk véletlenszerűen kiválasztani egy elemet, akkor a `choice()` függvény használható:

```
random.choice(['alma', 'körte', 'eper'])
```

Ez **egy véletlenszerű elemet ad vissza a megadott szekvenciából** (pl. lista).

Összefoglalva:

- Számokból: `randint(kisebb, nagyobb)`
- Listából: `choice(lista)`

Ne feledd: a modul használata előtt mindig importálni kell:

```
import random
```